



Lecture 3: K-Nearest Neighbors (KNN)

Introduction au Machine Learning

Prof. Aissam BERRAHOU

Département des systèmes embarqués et mobile

17 mars 2025

- Implémenter l'algorithme des k plus proches voisins.
- Calculer distances et similarités pour différents types de représentations des données.
- Définir la frontière de décision de l'algorithme du plus proches voisins.
- Expliquer pourquoi l'algorithme des k plus proches voisins est susceptible de ne pas marcher en haute dimension.

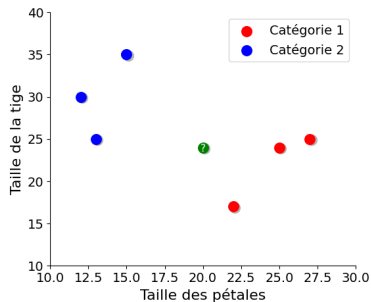
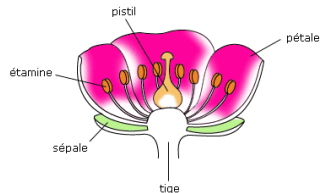
- Introduction
- Principe
- Implémentation avec Python
- Points forts et faibles de KNN
- TP

K-Nearest Neighbors (KNN) en français **k-voisins les plus proches**.

- Algorithme d'apprentissage automatique supervisé.
- Utilisé pour la classification et la régression.
- Prédiction sur des variables qualitatives et quantitatives.
- Méthode non paramétrique.

Introduction

T. pétales	T. tige	Catégorie
22	17	1
27	25	1
25	24	1
15	35	2
13	25	2
12	30	2
20	24	?



Algorithm 1 KNN

Entrée:

E : Base d'apprentissage (ensemble de données étiquetées),
 x : Point de données inconnu à classer ou prédire
 T : Type d'application (Classification ou Régression)
 k : Nombre de voisins plus proches

Sortie:

R : Résultat (classe prédite ou valeur de régression)

Pour chaque point de données $e_i \in E$ **Faire**

Calculer la distance $d(x, e_i)$ entre x et e_i (Ex :distance euclidienne).

Stocker la paire $(d(x, e_i), e_i)$ dans une liste **distances**.

Fin Pour

Trier la liste **distances** par ordre croissant de distance.

Sélectionner les k premiers éléments de la liste triée (les k plus proches voisins).

Extraire les étiquettes (ou valeurs) des k voisins sélectionnés.

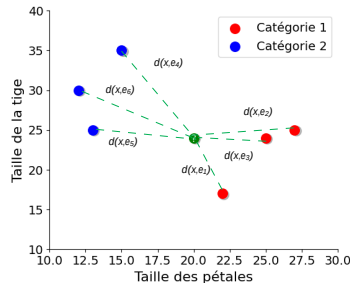
Si $T == \text{Classification}$ **Alors**

Renvoyer $R =$ l'étiquette majoritaire parmi les k voisins.

Sinon

Renvoyer $R =$ la moyenne des valeurs des k voisins.

Fin Si



Application de l'algorithme : classifier $x(20, 24)$ pour $k = 3$

Observations	T. pétales	T. tige	Catégorie
e1	22	17	1
e2	27	25	1
e3	25	24	1
e4	15	35	2
e5	13	25	2
e6	12	30	2

Application de l'algorithme : classifier $x(20, 24)$ pour $k = 3$

Pour chaque point de données $e_i \in E$ **Faire**

Calculer la distance $d(x, e_i)$ entre x et e_i (par exemple, distance euclidienne).

Stocker la paire $(d(x, e_i), e_i)$ dans une liste **distances**.

Fin Pour

Observations	T. pétales	T. tige	Catégorie	Distance(e_i, x)
e1	22	17	1	7.28
e2	27	25	1	7.07
e3	25	24	1	5
e4	15	35	2	12.08
e5	13	25	2	7.07
e6	12	30	2	10

Application de l'algorithme : classifier $x(20, 24)$ pour $k = 3$

Trier la liste **distances** par ordre croissant de distance.

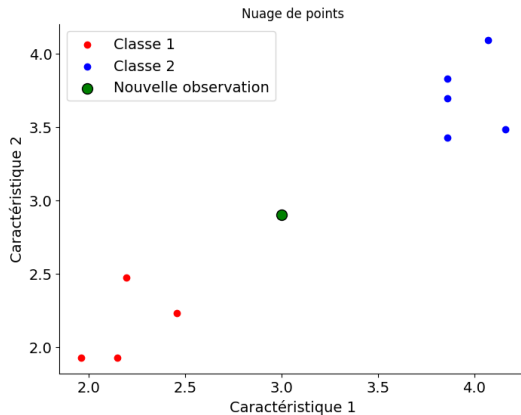
Sélectionner les k premiers éléments de la liste triée (les k plus proches voisins).

Extraire les étiquettes (ou valeurs) des k voisins sélectionnés.

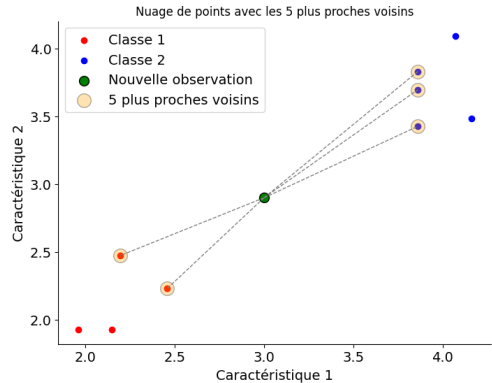
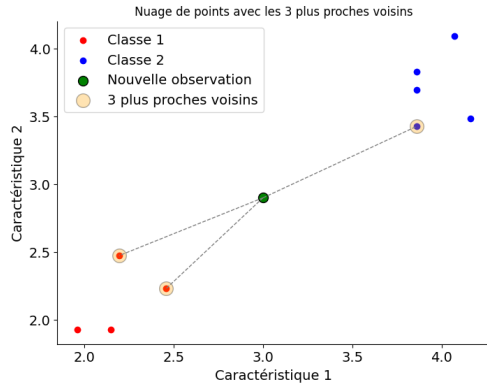
Observations	T. pétales	T. tige	Catégorie	Distance(e_i, x)
e3	25	24	1	5
e2	27	25	1	7.07
e5	13	25	2	7.07
e1	22	17	1	7.28
e6	12	30	2	10
e4	15	35	2	12.08

Pour $k = 3$, la nouvelle observation x est classée 1.

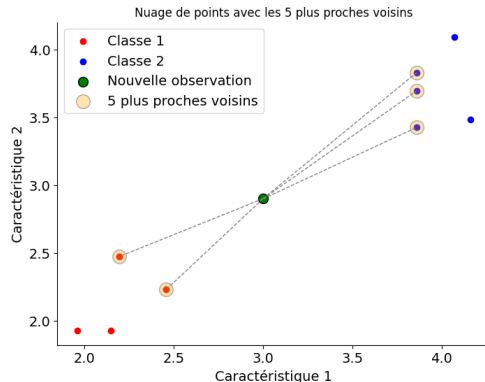
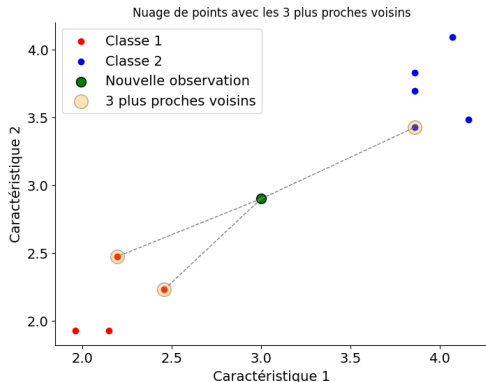
Application de l'algorithme : classifier $x(20, 24)$ pour $k = 3$



Application de l'algorithme : classifier $x(20, 24)$ pour $k = 3$



Application de l'algorithme : Classifier $x(20, 24)$ pour $k = 3$



Comment choisit-on la valeur optimale du paramètre k ?

Comment choisit-on la valeur optimale du paramètre k ?

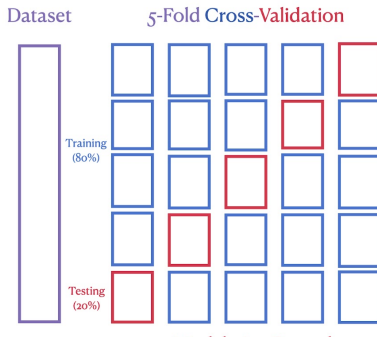
- Il est préférable de sélectionner une valeur K impaire.
- Le choix de k est crucial pour les performances de l'algorithme :
 - k trop petit : Sensible au bruit et aux outliers (surapprentissage).
 - k trop grand : Peut inclure des points trop éloignés, réduisant la précision (sous-apprentissage).

Comment choisit-on la valeur optimale du paramètre k ?

- Méthodes pour choisir k :
 - Validation croisée.
 - Méthode ELbow(Coude).

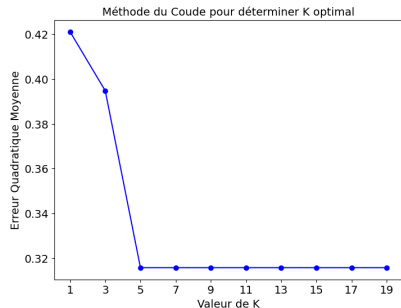
Validation croisée : consiste à diviser le jeu de données en plusieurs sous-ensembles ("folds") et à évaluer les performances du modèle pour différentes valeurs de K .

1. Diviser le jeu de données en k folds (par exemple, 5 ou 10).
2. Pour chaque valeur de K (par exemple, de 1 à 20) :
 - Entraîner le modèle sur $k-1$ folds.
 - Évaluer le modèle sur le fold restant.
 - Calculer la moyenne des performances (par exemple, la précision ou l'erreur).
3. Choisir la valeur de K qui donne la meilleure performance moyenne.



Méthode ELbow(Coude) : consiste à tracer l'erreur du modèle (par exemple, l'erreur de classification ou l'erreur quadratique moyenne) en fonction de différentes valeurs de K . La valeur optimale de K est souvent celle où l'erreur commence à se stabiliser (le "coude" de la courbe).

1. Tester différentes valeurs de K .
2. Calculer l'erreur pour chaque valeur de K .
3. Tracer l'erreur en fonction de K .
4. Choisir la valeur de K au niveau du "coude".



Distances :

- Distance euclidienne :

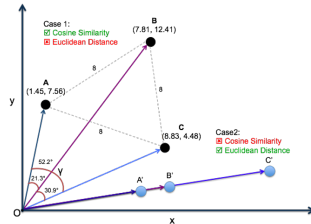
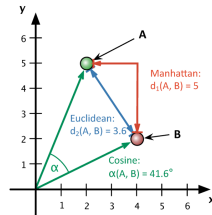
$$d_e(U, V) = \sqrt{\sum_{i=1}^n (U_i - V_i)^2}$$

- Distance de Manhattan :

$$d_m(U, V) = \sum_{i=1}^n |U_i - V_i|$$

- Distance cosinus :

$$\begin{aligned} d_c(U, V) &= 1 - \frac{\sum_{i=1}^n U_i V_i}{\sqrt{\sum_{i=1}^n U_i^2} \sqrt{\sum_{i=1}^n V_i^2}} \\ &= 1 - \frac{U \times V^T}{\|U\| \times \|V\|} \\ &= 1 - \cos(\alpha) \end{aligned}$$



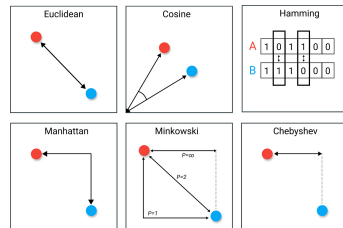
Distances :

- **Distance de Minkowski** : une généralisation des distances euclidienne et de Manhattan. Elle est définie par :

$$d(U, V) = \left(\sum_{i=1}^n |U_i - V_i|^p \right)^{1/p}$$

où p est un paramètre appelé ordre de la distance.

- Si $p=1$ on obtient la distance de Manhattan.
- Si $p=2$, on obtient la distance euclidienne.



```
from sklearn.neighbors import KNeighborsClassifier
from sklearn.model_selection import train_test_split
from sklearn.datasets import load_iris
# Loading data
irisData = load_iris()
# Create feature and target arrays
X = irisData.data
y = irisData.target
# Split into training and test set
X_train, X_test, y_train, y_test = train_test_split( X, y, test_size = 0.2,
    ↪ random_state=42)
knn = KNeighborsClassifier(n_neighbors=7) # k = 7
knn.fit(X_train, y_train)
# Calculate the accuracy of the model
print("Accuracy:", knn.score(X_test, y_test))
```

Points forts.

- ✓ K-NN est un algorithme assez simple à comprendre.
- ✓ K-NN effectue des prédictions juste à temps (à la volée) en calculant la similarité entre une observation en entrée et les différentes observations de la base de données.



Points faibles.

- ✗ Coûteux en calcul pour de grands ensembles de données.
 - ✗ KNN se base principalement sur le choix de la méthode de calcul de la distance ainsi que le nombre de voisins k , qui peut ne pas être évident. Il faut essayer plusieurs combinaisons et faire du tuning de l'algorithme pour avoir un résultat satisfaisant.
- Coûteux en calcul pour de grands ensembles de données.



1. Implémenter KNN from scratch sans utiliser `scikit-learn`.
2. Appliquer KNN à des problèmes concrets sur des bases de données.
3. Comparer les performances de KNN avec d'autres modèles (Régression Logistique, SVM, Arbres de décision).
4. Explorer des sujets inédits basés sur des données ouvertes et actuelles.